

PLSQL

Sanae MAZOUZ

Introduction

- **Le langage Oracle PL SQL est un langage procédural et est une extension procédurale du langage SQL. Il permet de grouper des traitements et de les soumettre au noyau en un bloc unique de traitement.**
- **Le langage SQL est non procédural alors que le PL SQL est un langage procédural.**
- **Le PL SQL sert à programmer des procédures, des fonctions, des triggers, des packages.**

Introduction

PL/SQL peut être utilisé sous trois formes :

- **Un bloc de code, exécuté comme une commande SQL, via un interpréteur standard (SQL*plus ou SQL Developer)**
- **Un fichier de commande PL/SQL.**
- **Un programme stocké (procédure, fonction, package ou trigger).**

Avantages

Les principaux avantages de PL/SQL sont :

☐ La modularité :

- **Un bloc d'instruction peut être composé d'un autre.**
- **Un bloc peut être nommé pour devenir une procédure ou une fonction cataloguée → réutilisable.**
- **Une procédure, ou fonction, cataloguée peut être incluse dans un paquetage (*package*) pour mieux contrôler et réutiliser ces composants logiciels.**

Avantages

❑ La portabilité :

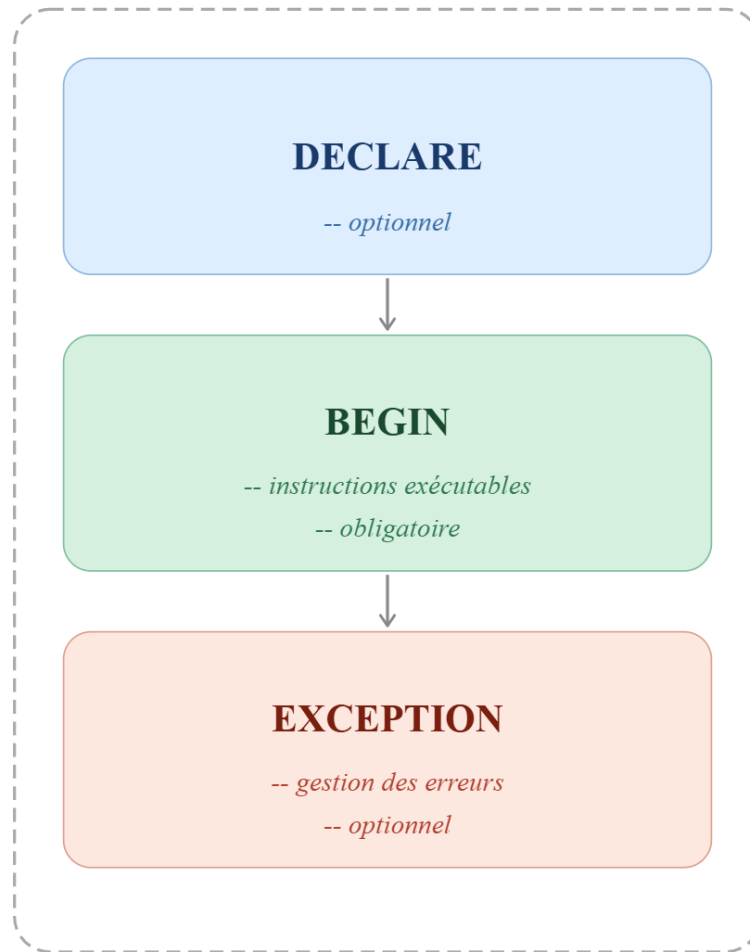
- **Un programme PL/SQL est indépendant du système d'exploitation qui héberge le serveur Oracle.**
- **En changeant de système, les applicatifs n'ont pas à être modifiés.**

Structure d'un programme

Un programme PL/SQL est composé de trois sections

- **DECLARE** (section optionnelle)
 - *déclare les variables, types, curseurs, exceptions, etc.*
- **BEGIN** (section obligatoire)
 - contient le code PL/SQL incluant ou non des directives SQL (jusqu'à l'instruction END;).
 - Le caractère « / » termine un bloc pour son exécution dans l'interface SQL*Plus.
- **EXCEPTION** (section optionnelle)
 - *permet de traiter les erreurs retournées par le SGBD à la suite d'exécutions d'instructions SQL.*

Structure d'un programme



Sous-bloc

- Un bloc peut être imbriqué dans le code d'un autre bloc (on parle de sous-bloc).
- Un sous-bloc peut aussi se trouver dans la partie des exceptions.
- Un sous-bloc commence par **BEGIN** et se termine par **END**.
- Un objet déclaré dans un sous-bloc n'est pas visible du bloc supérieur (principe des accolades de C et Java).

Sous-bloc

```
DECLARE    --section optionnelle
           déclaration variables, constantes, types, curseurs,...

BEGIN      --section obligatoire
           contient le code PL/SQL
           DECLARE    --section optionnelle
                   déclaration variables, constantes, types, curseurs,...

           BEGIN      --section obligatoire
                   contient le code PL/SQL

           EXCEPTION  --section optionnelle
                   traitement des erreurs

           END;        --obligatoire
EXCEPTION  --section optionnelle
           traitement des erreurs

END;        --obligatoire
```

Jeu de caractères

- ❑ Les programmes PL/SQL sont capables d'interpréter les caractères suivants :
 - Lettres A à Z et a à z ;
 - Chiffres de 0 à 9 ;
 - Symboles () + - * / < > = ! ~ ^ ; : . ' @ % , " # \$ & _ | { } ? [] ;
 - Tabulations, espaces et retours-chariot.
- ❑ PL/SQL n'est pas sensible à la casse (*not case sensitive*).

Identificateurs

- Un identificateur commence par une lettre suivie (optionnel) de symboles (lettres, chiffres, \$, _, #).
- Un identificateur peut contenir jusqu'à trente caractères.
- Les autres signes (pourtant connus du langage) sont interdits.

Identificateurs

Autorisés

X

t2

téléphone#

code_brevet

codeBrevet

oracle\$nombre

Interdits

moi&toi (symbole &)

debit-credit (symbole -)

on/off (symbole /)

code brevet (espace)

Commentaires

PL/SQL supporte deux types de commentaires :

- ***monolignes*** : commençant par le symbole **--** et finissant à la fin de la ligne.

Sur une ligne

```
-- Lecture de la table  
SELECT salaire INTO v_salaire  
FROM Pilote      -- Extraction du salaire  
WHERE nom = 'Thierry Albaric';  
v_bonus := v_salaire * 0.15;      -- Calcul
```

Commentaires

- **Multilignes** : commençant par `/*` et finissant par `*/`.

Sur plusieurs lignes

```
/* Lecture de la table Pilote */  
SELECT salaire INTO v_salaire  
FROM Pilote  
    /* Extraction du salaire  
       pour calculer le bonus */  
WHERE nom = 'Thierry Albaric';  
v_bonus := v_salaire * 0.15;      /*Calcul*/
```

Variables

- **Un programme PL/SQL est capable de manipuler des variables et des constantes (dont la valeur est invariable).**

Les variables et les constantes sont déclarées (et éventuellement initialisées) dans la section DECLARE.

Variables

- Plusieurs types de variables sont manipulés par un programme PL/SQL:
 - Scalaires recevant une seule valeur d'un type SQL (exemple : colonne d'une table) ;
 - Composites (%ROWTYPE, RECORD et TYPE) ;
 - Références (REF) .

Variables scalaires

La déclaration d'une variable scalaire est de la forme suivante :

- **CONSTANT** précise qu'il s'agit d'une constante (empêche toute modification de la variable ;
- **NOT NULL** pose une contrainte en ligne sur la variable ;
- **DEFAULT** permet d'initialiser la variable (équivalent à l'affectation :=).

identificateur [CONSTANT] typeDeDonnée [NOT NULL] [:= | DEFAULT expression];

Exemples

Variables	Constantes et expressions
<pre>DECLARE v_dateNaissance DATE; /* équivaut à v_dateNaissance DATE:= NULL; */ v_capacité NUMBER(3) := 999; v_téléphone CHAR(14) NOT NULL := '06-76-85-14-89'; v_trouvé BOOLEAN NOT NULL := TRUE; BEGIN ... </pre>	<pre>DECLARE c_pi CONSTANT NUMBER := 3.14159; v_rayon NUMBER := 1.5; v_aire NUMBER := c_pi * v_rayon**2; -- v_groupeSanguin CHAR(3) := 'O+'; /* équivaut à v_groupeSanguin CHAR(3) DEFAULT 'O+'; */ v_dateValeur DATE := SYSDATE + 2; BEGIN ... </pre>

Variables scalaires

Remarque :

Il n'est pas possible d'affecter **une valeur nulle** à une variable définie **NOT NULL** (*l'erreur renvoyée est l'exception prédéfinie VALUE_ERROR*).

La contrainte **NOT NULL** doit être suivie d'une clause d'initialisation.

Restrictions

- Il est impossible d'utiliser un identificateur dans une expression s'il n'est pas déclaré au préalable.
- Ici, la déclaration de la variable maxi est incorrecte

```
DECLARE
```

```
maxi NUMBER := 2 * mini;
```

```
mini NUMBER := 15;
```

Erreur de déclaration de la variable mini

Restrictions

- À l'inverse de la plupart des langages récents, les déclarations multiples ne sont pas permises.
- Celle qui suit est incorrecte :

```
DECLARE  
i, j, k NUMBER;
```

Erreur de déclaration des variables

Conventions recommandées

Objet	Convention	Exemple
Variable	<i>v_nomVariable</i>	v_compteur
Constante	<i>c_nomConstante</i>	c_pi
Exception	<i>e_nomException</i>	e_pasTrouvé
Type RECORD	<i>nomRecord_rec</i>	employe_rec
Variable RECORD	<i>r_nomVariable</i>	r_employe
Variable ROWTYPE	<i>rty_nomVariable</i>	rty_employe
Type-tableau	<i>nomTypeTableau_tytabs</i>	employes_tytabs
Variable tableau	<i>tab_nomTableau</i>	tab_employes
Curseur	<i>nomCurseur_cur</i>	employes_cur
Variable de substitution (SQL*Plus)	<i>s_nomVariable</i>	s_salaire
Variable de session (globale)	<i>g_nomVariable</i>	g_salaire

Types scalaires				
BINARY_INTEGER	DEC	DECIMAL	DOUBLE PRECISION	FLOAT
INT	INTEGER	NATURAL	NATURALN	NUMBER
NUMERIC	PLS_INTEGER	POSITIVE	POSITIVEN	REAL
SIGNTYPE	SMALLINT	CHAR	CHARACTER	LONG
LONG RAW	NCHAR	NVARCHAR2	RAW	ROWID
STRING	UROWID	VARCHAR	VARCHAR2	
BOOLEAN	DATE			
INTERVAL DAY TO SECOND (9i)	INTERVAL YEAR TO MONTH (9i)	TIMESTAMP (9i)	TIMESTAMP WITH LOCAL TIME ZONE (9i)	TIMESTAMP WITH TIME ZONE (9i)

Types composés		
RECORD	TABLE	VARRAY

Types références	
REF CURSOR	REF type_objet

Types grands objets			
BFILE	BLOB	CLOB	NCLOB

Types supplémentaires				
SYS.ANYDATA	SYS.ANYTYPE	SYS.ANYDATASET		
XMLTYPE	URITYPE			
MDSYS.SDO_GEOMETRY				
ORDSYS.ORDAUDIO	ORDSYS.ORDIMAGE	ORDSYS.ORDVIDEO	ORDSYS.ORDDOC	ORDSYS.ORDIMAGESIGNATURE

Les variables (types caractères)

- **CHAR(n)**: Chaîne de caractères de longueur fixe avec n compris entre 1 et 32767 (par défaut 1).
- **VARCHAR2(n)**: Chaîne de caractères de longueur variable avec n compris entre 1 et 32767.
- **LONG**: Chaîne de caractères de longueur variable avec au maximum 32760 octets.
- **RAW(n)** : Chaîne de caractères ou données binaires de longueur variable avec n compris entre 1 et 32767. Le contenu d'une variable de ce type n'est pas interprété par PL/SQL (pas de gestion des caractères nationaux).

Les variables (types caractères)

- **LONG RAW**: Identique au type LONG qui peut contenir des données binaires Jeux de caractères multi-octets.
- **NCHAR(n)** : Chaîne de caractères de longueur fixe avec n compris entre 1 et 32767 (par défaut 1).
- **NVARCHAR2(n)** : Chaîne de caractères de longueur variable avec n compris entre 1 et 32767. Le nombre de caractères réellement stockés dépend du nombre d'octets utilisés pour coder chaque caractère.
- **UROWID, ROWID** : Permet de stocker l'adresse absolue d'une ligne dans une table sous la forme d'une chaîne de caractères.

Les variables (types numériques)

- ***NUMBER[(e,d)]*** : Nombre réel avec e chiffres significatifs stockés et d décimales.
- ***BINARY_INTEGER*** : Nombre entier compris entre $-2^{147\ 483\ 647}$ et $+2^{147\ 483\ 647}$ (Utilise les fonctions de la librairie arithmétique).
- ***PLS_INTEGER*** : Nombre entier compris entre $-2^{147\ 483\ 647}$ et $+2^{147\ 483\ 647}$ (Plus rapide que BINARY_INTEGER car il utilise les registres du processeur).

Les variables (Types pour les grands objets)

- **BFILE** : Stocke la référence vers un fichier du système d'exploitation
- **BLOB** : Permet de stocker un objet binaire jusqu'à 4 Go
- **CLOB** : Permet de stocker un ensemble de caractères, jusqu'à 4 Go
- **NCLOB** : Permet de stocker un ensemble de caractères, codés sur un ou plusieurs octets, jusqu'à 4 Go

Variables %TYPE

- La directive **%TYPE** déclare une variable selon la définition d'une colonne d'une table ou d'une vue existante.
- Elle permet aussi de déclarer une variable conformément à une autre variable précédemment déclarée.
- Il faut faire préfixer la directive **%TYPE** avec le nom de la table et celui de la colonne.

(identificateur1 nomTable.nomColonne%TYPE)

Variables %TYPE

- *ou avec le nom d'une variable existante*

(identificateur2 identificateur1%TYPE)

Variables %TYPE

Exemple :

Code PL/SQL

Commentaires

DECLARE

v_empno emp.empno%TYPE;

v_empno prend le type de la colonne empno de la table emp.

v_sal NUMBER(7,2) := 3000.00;

v_sal_min prend le type de la variable

v_sal_min v_sal%TYPE := v_sal*0.9;

v_sal et est initialisée à 2700,00.

BEGIN

...

Variables %ROWTYPE

- La directive **%ROWTYPE** permet de travailler au niveau d'un enregistrement (*record*).
- Il est composé d'un ensemble de colonnes.
- L'enregistrement peut contenir toutes les colonnes d'une table ou seulement certaines.
- Il est aussi possible d'insérer dans une table ou de modifier une table en utilisant une variable du type %ROWTYPE.

Code PL/SQL	Commentaires
<pre> DECLARE rty_emp emp%ROWTYPE; v_job emp.job%TYPE; BEGIN SELECT * INTO rty_emp FROM emp WHERE empno = 7369; v_job := rty_emp.job; -- ... rty_emp.empno := 8000; rty_emp.ename := 'CLARK'; -- ... INSERT INTO emp VALUES rty_emp; END; </pre>	La structure rty_emp est composée de toutes les colonnes de la table emp.
	Chargement de l'enregistrement rty_emp à partir d'une ligne (employé 7369). Accès à une valeur par la notation pointée.
	Insertion dans la table emp directement à partir de l'enregistrement.

Variables %ROWTYPE

Remarque :

Les colonnes récupérées par la directive **%ROWTYPE** n'héritent pas des contraintes NOT NULL qui seraient éventuellement déclarées au niveau de la table.

Variables RECORD

- Alors que la directive *%ROWTYPE* permet de déclarer une structure composée de colonnes de tables, elle ne convient pas à des structures de données personnalisées.
- Pour cela, nous avons le type de données RECORD :
 - Disponible depuis la version 7.
 - Définit vos propres structures de données.

Variables RECORD

- La syntaxe générale pour déclarer un **RECORD** est la suivante :

```
TYPE nomRecord IS RECORD  
( nomChamp typeDonnées [[NOT NULL] {:= | DEFAULT}  
  expression]  
[,nomChamp typeDonnées... ]... );
```

Exemple

Code PL/SQL	Commentaires
<pre>DECLARE TYPE employe_rec IS RECORD (empno NUMBER(4), ename CHAR(10), job CHAR(9) := 'CLERK', sal NUMBER(7,2)); r_emp1 employe_rec; r_emp2 employe_rec; BEGIN</pre>	Déclaration du RECORD contenant quatre champs ; initialisation du champ job par défaut. Déclaration de deux variables de type RECORD.
<pre> r_emp1.empno := 7369; r_emp1.ename := 'SMITH'; r_emp1.sal := 800.00; r_emp2 := r_emp1;</pre>	Initialisation des champs d'un RECORD. Affectation d'un RECORD.
<pre>...</pre>	

Variables RECORD

- Les types RECORD ne peuvent pas être stockés dans une table.
- Il est possible qu'un champ d'un RECORD soit lui-même un RECORD, ou soit déclaré avec les directives %TYPE ou %ROWTYPE.
- Les RECORD ne peuvent pas être comparés (nullité, égalité et inégalité).

Exemple

```
DECLARE
TYPE employe_rec IS RECORD
    (empno    NUMBER(4) ,
     ename     CHAR(10) ,
     job       CHAR(9)  := 'CLERK' ,
     sal       NUMBER(7,2)) ;

TYPE contrat_rec IS RECORD
    (r_employe    employe_rec,
     dateEmbauche  DATE,
     rty_coEmp     Emp%ROWTYPE,
     deptnom       Dept.dname%TYPE) ;
```

Variables tableaux (type TABLE)

- Les variables de type TABLE (*associative arrays*) *permettent de définir et de manipuler des tableaux dynamiques* (car définis sans dimension initiale).
- Un tableau est composé :
 - d'une clé primaire (de type BINARY_INTEGER)
 - et d'une colonne (de type scalaire, TYPE, ROWTYPE ou RECORD) pour stocker chaque élément.

Variables tableaux (type TABLE)

Syntaxe :

La syntaxe générale pour déclarer un type de tableau et une variable tableau est la suivante :

```
TYPE nomTypeTableau IS TABLE OF  
{typeScalaire | variable%TYPE | table.colonne%TYPE |  
table.%ROWTYPE};  
nomTableau nomTypeTableau;
```

Code PL/SQL	Commentaires
<pre> DECLARE TYPE empnos_tytabs IS TABLE OF NUMBER(4) INDEX BY BINARY_INTEGER; TYPE enames_tytabs IS TABLE OF Emp.ename%TYPE INDEX BY BINARY_INTEGER; TYPE emps_tytabs IS TABLE OF Emp%ROWTYPE INDEX BY BINARY_INTEGER; tab_empnos empnos_tytabs; tab_enames enames_tytabs; tab_emps emps_tytabs; </pre>	NUMBER(4) numéros d'employés %TYPE type ancré sur Emp.ename %ROWTYPE ligne complète de Emp Déclaration des tableaux.
<pre> BEGIN tab_empnos(1) := 7369; tab_empnos(2) := 7499; tab_enames(7369) := 'SMITH'; tab_enames(7499) := 'ALLEN'; tab_emps(0).empno := 7369; tab_emps(0).ename := 'SMITH'; END; </pre>	Initialisations. 1, 2... Convention classique PL/SQL. empno Accès direct par clé métier. 0, 1... Convention informatique.

Fonctions pour les tableaux

- **PL/SQL propose un ensemble de fonctions qui permettent de manipuler des tableaux.**
- **Il n'est pas possible actuellement d'appeler une de ces fonctions dans une instruction SQL(SELECT, INSERT, UPDATE ou DELETE).**

Exemple

Code PL/SQL	Commentaires
<code>IF tab_emps.EXISTS(0) THEN...</code>	Renvoie « vrai » car il existe un élément à l'indice 0.
<code>v_nombre := tab_empnos.COUNT;</code>	La variable v_nombre contient 2.
<code>v_premier := tab_empnos.FIRST;</code> <code>v_dernier := tab_empnos.LAST;</code>	La variable v_premier contient 1, v_dernier contient 2.
<code>v_avant := tab_empnos.PRIOR(2);</code>	La variable v_avant contient 1.
<code>tab_empnos.DELETE;</code>	Suppression de tous les éléments de tab_empnos.

Résolution de noms

- Lors des conflits potentiels de noms (variables ou colonnes) dans des instructions SQL (principalement INSERT, UPDATE, DELETE et SELECT), le nom de la colonne de la table est prioritairement interprété au détriment de la variable (de même nom).

Résolution de noms

- Dans l'exemple suivant, l'instruction **DELETE** supprime tous les pilotes (et non pas seulement le pilote 'Pierre Lamothe'), car Oracle considère les deux identificateurs comme la colonne de la table et non pas comme deux variables différentes !

```
DECLARE
nom CHAR(20) := 'Pierre Lamothe';
BEGIN
DELETE FROM Pilote WHERE nom = nom;
...
```

Résolution de noms

- Pour résoudre ce problème, deux solutions existent.
- La première consiste à nommer toutes les variables explicitement et différemment des colonnes.
- La deuxième consiste à utiliser une étiquette de bloc (*block label*) pour lever les ambiguïtés.

Préfixer les variables	Étiquette de bloc
<pre>DECLARE v_nom CHAR(20) := 'Pierre Lamothe'; BEGIN DELETE FROM Pilote WHERE nom = v_nom; END;</pre>	<pre><<principal>> DECLARE nom CHAR(20) := 'Pierre Lamothe'; BEGIN DELETE FROM Pilote WHERE nom = principal.nom; END;</pre>

Opérateurs

- Les opérateurs SQL (logiques, arithmétiques, concaténation...) sont disponibles aussi avec PL/SQL.

Code PL/SQL	Commentaires
<pre>DECLARE v_compteur NUMBER(3) DEFAULT 0; v_booléen BOOLEAN; v_nombre NUMBER(3); BEGIN</pre>	
<pre> v_compteur := v_compteur+1;</pre>	Incrémentation, opérateur +
<pre> v_booléen := (v_compteur = v_nombre);</pre>	v_booléen reçoit NULL du fait de la condition.
<pre> v_booléen := (v_nombre IS NULL);</pre>	v_booléen reçoit TRUE car la condition est vraie.

Variables de substitution

Variables de substitution

- Variables passées en paramètres d'entrée d'un bloc PL/SQL des variables définies sous SQL*Plus.
- On accède aux valeurs d'une telle variable dans le code PL/SQL en faisant préfixer le nom de la variable du symbole « & ».
- La directive ACCEPT (lecture de la variable au clavier)
- **Il ne faut pas déclarer des variables de substitution.**

Exemple

Code PL/SQL

```
ACCEPT s_empno PROMPT 'Entrer numéro employé : '  
ACCEPT s_augment PROMPT 'Entrer augmentation sal : '  
DECLARE  
    v_ename Emp.ename%TYPE;  
    v_sal Emp.sal%TYPE;  
BEGIN  
    SELECT ename, sal INTO v_ename, v_sal  
        FROM Emp WHERE empno = '&s_empno';  
    v_sal := v_sal + &s_augment;  
    DBMS_OUTPUT.PUT_LINE('Nouveau salaire : '  
        || v_sal || ' de ' || v_ename);  
END;
```

Sous SQL*Plus

Entrer numéro
employé : 7369
Entrer augmentation
sal : 500

Nouveau salaire :
1300 de SMITH

Procédure PL/SQL
terminée avec succès.

Variables de substitution

Remarque importante :

Il faut exécuter le bloc à l'aide de la commande start ou @ et non pas par copier-coller d'un éditeur de texte vers la fenêtre SQL*Plus (à cause des instructions d'entrée ACCEPT).

Variables de session

- Il est possible de définir des variables de session (globales) définies sous SQL*Plus au niveau d'un bloc PL/SQL.
- La directive SQL*Plus à utiliser en début de bloc est VARIABLE.
- Dans le code PL/SQL, il faut faire préfixer le nom de la variable de session du symbole « : ».
- L'affichage de la variable sous SQL*Plus est réalisé par la directive PRINT.

Variables de session

Code PL/SQL

```
VARIABLE g_compteur NUMBER;  
DECLARE  
    v_compteur NUMBER(3) := 99;  
BEGIN  
    :g_compteur := v_compteur+1;  
END;
```

Sous SQL*Plus

```
SQL> PRINT g_compteur ;  
  
G_COMPTEUR  
-----  
100
```

Paquetage DBMS_OUTPUT

- Il est possible d'afficher sous SQL*Plus des résultats calculés par un bloc PL/SQL :
 - avec des variables de session.
 - En utilisant des procédures du paquetage DBMS_OUTPUT.
- Ce paquetage assure la gestion des entrées/sorties de blocs ou sous-programmes PL/SQL (fonctions et procédures cataloguées, paquetages ou déclencheurs).

Procédure	Explication
<code>ENABLE (taille_tampon IN INTEGER DEFAULT 2000)</code>	Activation du paquetage dans la session.
<code>DISABLE</code>	Désactivation du paquetage dans la session.
<code>PUT(ligne IN VARCHAR2 DATE NUMBER);</code>	Mise dans le tampon d'un paramètre.
<code>NEW_LINE(ligne OUT VARCHAR2, statut OUT INTEGER)</code>	Écriture du caractère fin de ligne dans le tampon.
<code>PUT_LINE(ligne IN VARCHAR2 DATE NUMBER);</code>	<code>PUT</code> puis <code>NEW_LINE</code> .
<code>GET_LINE(ligne OUT VARCHAR2, statut OUT INTEGER)</code>	Affectation d'une chaîne du tampon dans une variable.
<code>GET_LINES(tab OUT DBMS_OUTPUT.CHARARR, nombreLignes IN OUT INTEGER);</code> <code>CHARARR</code> table de <code>VARCHAR2(255)</code>	Affectation de chaînes du tampon dans un tableau.

Code PL/SQL	Commentaires
<pre> SET SERVEROUTPUT ON DECLARE v_nbrEmp NUMBER; BEGIN </pre>	Activation du paquetage sous SQL*Plus.
<pre> DBMS_OUTPUT.ENABLE; DBMS_OUTPUT.PUT_LINE('Nous sommes le : ' SYSDATE); DBMS_OUTPUT.PUT_LINE('La racine de 2 = ' SQRT(2)); SELECT COUNT(*) INTO v_nbrEmp FROM emp; DBMS_OUTPUT.PUT_LINE ('Il y a ' v_nbrEmp ' employes dans la table'); END; / </pre>	Activation du paquetage sous PL/SQL. Affichage.
<pre> Nous sommes le : 14/07/03 La racine de 2 = 1,41421356237309504880168872420969807857 Il y a 14 employes dans la table Procédure PL/SQL terminée avec succès. </pre>	Résultats.

La section exécution

- ***Initialisation des variables***

- ✓ Opérateur :=

- ou

- ✓ SELECT ... INTO ...

- ***Remarque***

le **SELECT** doit ramener **une et une seule ligne**, sinon erreur.

- ***Visibilité des variables***

Une variable est visible dans le bloc où elle a été déclarée et dans les blocs imbriqués si elle n'a pas été redéfinie.

Structures de contrôles

PL/SQL offre la possibilité de programmer :

- **les structures conditionnelles *si et cas (IF... et CASE) ;***
- **les structures répétitives *tant que, répéter et pour (WHILE, LOOP, FOR).***

La structure IF

Trois formes de IF

- On peut distinguer trois formes de structure IF :
 - IF-THEN
 - *IF-THEN-ELSE*
 - *IF-THEN-ELSIF*
- Les opérateurs dans les conditions sont du type = , < , <= , != , LIKE , BETWEEN , AND , OR ,...
- Il faut Noter « END IF » en fin de structure et non pas « ENDIF ».

La structure IF

IF-THEN	IF-THEN-ELSE	IF-THEN-ELSIF
<pre>IF condition THEN instructions; END IF;</pre>	<pre>IF condition THEN instructions; ELSE instructions; END IF;</pre>	<pre>IF condition1 THEN instructions; ELSIF condition2 THEN instructions; ELSE instructions; END IF;</pre>
<pre>DECLARE v_téléphone CHAR(14) NOT NULL := '06-76-85-14-89'; BEGIN IF SUBSTR(v_téléphone,1,2)='06' THEN DBMS_OUTPUT.PUT_LINE ('C'est un portable!'); ELSE DBMS_OUTPUT.PUT_LINE ('C'est un fixe...'); END IF; END;</pre>		

Conditions booléennes

- Les variables booléennes peuvent prendre trois valeurs (TRUE, FALSE, NULL).
- Il est à noter que la négation de NULL (NOT NULL) renvoie une valeur nulle.

AND	TRUE	FALSE	NULL
TRUE	TRUE	FALSE	NULL
FALSE	FALSE	FALSE	FALSE
NULL	NULL	FALSE	NULL

OR	TRUE	FALSE	NULL
TRUE	TRUE	TRUE	TRUE
FALSE	TRUE	FALSE	NULL
NULL	TRUE	NULL	NULL

La structure CASE

- Comme l'instruction IF, la structure CASE permet d'exécuter une séquence d'instructions en fonction de différentes conditions.
- La structure CASE est utile lorsqu'il faut évaluer une même expression et proposer plusieurs traitements pour diverses conditions.
- La clause ELSE est optionnelle. Si elle n'est pas présente, PL/SQL ajoute par défaut l'instruction :
« ELSE RAISE CASE_NOT_FOUND » .

La structure CASE

CASE

[<<étiquette>>]

CASE *variable*

WHEN *expr1* THEN *instructions1*;

WHEN *expr2* THEN *instructions2*;

...

WHEN *exprN* THEN *instructionsN*;

[ELSE *instructionsN+1*];

END CASE [*étiquette*];

searched CASE

[<<étiquette>>]

CASE

WHEN *condition1* THEN *instructions1*;

WHEN *condition2* THEN *instructions2*;

...

WHEN *conditionN* THEN *instructionsN*;

[ELSE *instructionsN+1*];

END CASE [*étiquette*];

```
DECLARE
```

```
  v_mention CHAR(2);
```

```
  v_note    NUMBER(4,2) := 9.8;
```

```
BEGIN
```

```
  ...
```

```
IF v_note >= 16 THEN
```

```
  v_mention := 'TB';
```

```
ELSIF v_note >= 14 THEN
```

```
  v_mention := 'B';
```

```
ELSIF v_note >= 12 THEN
```

```
  v_mention := 'AB';
```

```
ELSIF v_note >= 10 THEN
```

```
  v_mention := 'P';
```

```
ELSE
```

```
  v_mention := 'R';
```

```
END IF;
```

```
END;
```

```
/
```

```
CASE
```

```
  WHEN v_note >= 16 THEN
```

```
    v_mention := 'TB';
```

```
  WHEN v_note >= 14 THEN
```

```
    v_mention := 'B';
```

```
  WHEN v_note >= 12 THEN
```

```
    v_mention := 'AB';
```

```
  WHEN v_note >= 10 THEN
```

```
    v_mention := 'P';
```

```
ELSE
```

```
  v_mention := 'R';
```

```
END CASE;
```

```
END;
```

```
/
```

Les structures répétitives

- Les structures répétitives sont :
 - *WHILE*
 - *WHEN*
 - *FOR*
- Utilisent l'instruction :

```
LOOP...  
.....  
END LOOP
```

La structure répétitive: *WHILE*

- *Avant chaque itération*, la condition est évaluée. Si elle est vraie, la séquence d'instructions est exécutée, puis la condition est réévaluée pour un éventuel nouveau passage dans la boucle.
- Ce processus continue jusqu'à ce que la condition soit fausse pour passer en séquence après le ***END LOOP***.

Syntaxe :

```
WHILE condition  
LOOP  
  instructions;  
END LOOP;
```

Exemple

Calculer le reste de la division de 5432 par 5.

```
SET SERVEROUTPUT ON
DECLARE
reste NUMBER := 5432;
BEGIN
    WHILE reste >= 5
    LOOP
        reste := reste -5;
    END LOOP;
    dbms_output.put_line('La valeur de a est:' || reste);
END;

/
```

La structure répétitive : WHEN

- EXIT force la sortie de la boucle sans conditions.
- EXIT WHEN permet une sortie de boucle si la condition est vraie.
- EXIT <<label>> WHEN permet une sortie d'une boucle nommée label si la condition est vraie.
- EXIT <<label>> force une sortie de boucle nommée label.

Syntaxe :

```
<<label>>  
LOOP  
    instruction1;  
    instruction2;  
EXIT [label][WHEN condition1]  
END LOOP label;
```

Exemple

Insérer les 10 premiers chiffres dans la table result.

```
DECLARE
    nb      NUMBER := 1;
BEGIN
    LOOP
        INSERT INTO result VALUES (nb);
        nb := nb + 1;
        EXIT WHEN nb > 10;
    END LOOP;
END ;
```

La structure répétitive : FOR

Dans un traitement itératif la structure FOR IN permet de boucler grâce à un nombre d'itérations connu à l'avance.

```
FOR indice IN [REVERSE] exp1 .. exp2
  LOOP
    instructions;
  END LOOP;
```

Remarque :

- ✓ Inutile de déclarer l'indice
- ✓ Indice varie de exp1 à exp2 de 1 en 1
- ✓ Si REVERSE est précisé, indice varie de exp2 à exp1 avec un pas de -1.

Exemple

Calcul de la factorielle de 5

```
DECLARE
    fact NUMBER := 1;
BEGIN
    FOR i IN 1 .. 5
    LOOP
        fact := fact * i ;
    END LOOP;
END;
```

Interactions simples avec la base

Extraire des données : Erreurs

la requête **SELECT INTO** doit renvoyer impérativement 1 seul enregistrement, sinon PL / SQL déclenche une exception:

- Si le **select** renvoie plus d'une ligne, une exception « **TOO_MANY_ROWS** » (**ORA-01422**) est levée.
- Si le select ne renvoie aucune ligne, une exception « **NO_DATA_FOUND** » (**ORA-01403**) est levée.

Interactions simples avec la base

Modification de données

- Les requêtes SQL (**insert**, **update**, **delete**,...) peuvent utiliser les variables PL/SQL.
- Les **commit** et **rollback** doivent être explicites ; aucun changement n'est effectué automatiquement à la sortie d'un bloc.

Interactions simples avec la base

Exemple :

```
DECLARE
```

```
    v_emp emp%ROWTYPE;
```

```
BEGIN
```

```
    select * into v_emp from emp where ename = 'KING';
```

```
    v_emp.empno := v_emp.empno + 5;
```

```
    v_emp.ename := 'SMITH';
```

```
    INSERT INTO emp values v_emp;
```

```
END;
```

Les curseurs

- **Curseur** : zone de mémoire de taille fixe, utilisée par le noyau d'Oracle pour analyser et interpréter tout ordre SQL.
- Deux types de curseurs :
 - ✓ **Implicite** : créés et gérés par Oracle à chaque ordre SQL (lorsque la close **INTO** accompagne le **SELECT**).
 - ✓ **Explicite** : créés et gérés par l'utilisateur afin de pouvoir traiter un **SELECT** qui retourne **plusieurs tuples**.

Les curseurs

- Les curseurs sont définis dans la section DECLARE d'un bloc PL/SQL.
- Pour utiliser un curseur explicite, on doit passer par les étapes suivantes :
 1. Déclaration du curseur,
 2. Ouverture du curseur,
 3. Traitement des lignes du résultat,
 4. Fermeture du curseur.

Les curseurs

Déclaration du curseur

- Association d'un nom de curseur à une requête SELECT.
- Se fait dans la section DECLARE d'un bloc PL/SQL.

```
DECLARE  
  
CURSOR nom_curseur IS  
  
Requête_Select ;  
  
.....
```

Exemple

Soit la table employe (id, nomemp, sal, ville, ...)

```
DECLARE
```

```
CURSOR emp_rabat IS
```

```
SELECT nomemp, sal FROM employe WHERE ville =  
'Rabat';
```

```
.....
```

Les curseurs

Ouverture du curseur

- Alloue un espace mémoire au curseur et positionne les éventuels verrous.
- Après avoir déclaré le curseur, il faut l'ouvrir dans la section exécutable **BEGIN**.
- L'ouverture du curseur **amorce** l'analyse de SELECT et son exécution.
- La réponse est calculée et rangée dans un espace temporaire.

```
OPEN nom_curseur ;
```

Exemple

```
DECLARE
  CURSOR emp_rabat IS
    SELECT nomemp, sal FROM employe
    WHERE ville = 'Rabat';
BEGIN
  OPEN emp_rabat ;

  .....
END ;
```

Les curseurs

Traitement des lignes

- L'accès aux données se fait par la clause : **FETCH INTO**

```
FETCH nom_curseur INTO var1, var2, ...
```

- **FETCH** permet de récupérer une ligne de l'ensemble des lignes associés au curseur et stocke les valeurs dans des variables réceptrices.
- Pour traiter plusieurs tuples, il faut utiliser une boucle.

Les curseurs

Fermeture du curseur

- Après le traitement des lignes pour libérer la place mémoire, il faut fermer le curseur.

```
CLOSE nom_curseur ;
```

DECLARE

```
CURSOR emp_rabat IS SELECT nomemp, sal FROM  
employe WHERE ville = 'Rabat';  
nom employe.nomemp%TYPE ;  
salaire employe.sal%TYPE ;  
-- emp emp_rabat%ROWTYPE;
```

BEGIN

```
OPEN emp_rabat ;  
  LOOP  
    FETCH emp_rabat INTO nom, salaire ;  
    -- FETCH emp_rabat INTO emp;  
    ...  
    EXIT WHEN salaire > 10000;  
  END LOOP;  
CLOSE emp_rabat ;  
END;
```

Les curseurs

Statut d'un curseur (Attribut)

- Les attributs d'un curseur sont des indicateurs sur son état.
- Quatre attributs permettent d'évaluer l'état du curseur:
 - **%Found** : vrai si le dernier **FETCH** a ramené un tuple.
 - **%NotFound** : vrai si le dernier **FETCH** n'a ramené aucun tuple.
 - **%Rowcount** : compte le nombre de **FETCH** exécutés sur un curseur.
 - **%Isopen** : vrai si le curseur est ouvert.

DECLARE

```
CURSOR cur_emp IS
  SELECT empno, ename, sal
  FROM   emp
  WHERE  deptno = 10;
```

```
v_empno emp.empno%TYPE;
v_ename emp.ename%TYPE;
v_sal   emp.sal%TYPE;
totalSal NUMBER := 0;
```

BEGIN

```
OPEN cur_emp;
FETCH cur_emp
  INTO v_empno, v_ename, v_sal;
WHILE (cur_emp%FOUND) LOOP
  totalSal := totalSal + v_sal;
  FETCH cur_emp
    INTO v_empno, v_ename, v_sal;
END LOOP;
```

```
CLOSE cur_emp;
DBMS_OUTPUT.PUT_LINE(
  'Total sal dept 10 = '
  || totalSal);
```

END;

/

BEGIN

```
OPEN cur_emp;
LOOP
  FETCH cur_emp
    INTO v_empno, v_ename, v_sal;
  EXIT WHEN cur_emp%NOTFOUND;
  totalSal := totalSal + v_sal;
END LOOP;
```

```
CLOSE cur_emp;
```

```
DBMS_OUTPUT.PUT_LINE(
  'Total sal dept 10 = '
  || totalSal);
```

END;

/

Les curseurs

Avec un curseur implicite créé par Oracle, les mêmes attributs aux colonnes sont disponibles à condition de les préfixer par SQL. Ces attributs référents au dernier curseur implicite utilisé par l'application.

Curseur implicite	Curseur explicite
SQL%FOUND	nom-curseur%FOUND
SQL%NOTFOUND	nom-curseur%NOTFOUND
SQL%ISOPEN	nom-curseur%ISOPEN
SQL%ROWCOUNT	nom-curseur%ROWCOUNT

Les curseurs

Exemple

```
SET SERVEROUTPUT ON
    nb number;
BEGIN
    DELETE FROM emp WHERE ename='KING';
    nb := SQL%ROWCOUNT; -- curseur sql
    dbms_output.put_line('nb = ' || nb);
END;
```

Retourne le nombre d'enregistrement supprimés dont le champ nom est 'KING'.

Les curseurs

Utilisation simplifiée des curseurs

- L'utilisation **FOR LOOP** remplace **OPEN**, **FETCH** et **CLOSE**.
- Lorsque le curseur est invoqué, un enregistrement est automatiquement créé avec les mêmes éléments de données que ceux définies dans **SELECT**.
- La boucle **FOR** s'arrête d'elle-même à la fin de l'extraction de la dernière ligne du curseur.

Les curseurs

Simplification d'écriture

- L'attribut **%ROWTYPE** permet de déclarer une variable de même type que l'enregistrement (la ligne) de la table.
- Les éléments de la structure (nom_variable) sont identifiés par: **nom_variable.nom_colonne**.

Syntaxe :

```
DECLARE  
  CURSOR nom_curseur IS Requete_SELECT ;  
  nom_variable  nom_curseur%ROWTYPE ;
```

Les curseurs

```
DECLARE
CURSOR nom_curseur IS SELECT...;
nom_variable
nom_curseur%ROWTYPE;
BEGIN
OPEN nom_curseur;
LOOP
FETCH nom_curseur INTO
nom_variable;
EXIT WHEN nom_curseur%NOTFOUND;
...
END LOOP;
CLOSE nom_curseur;
END;
```

Ecriture complète

```
DECLARE
CURSOR nom_curseur IS SELECT ...;
BEGIN
FOR nom_variable IN nom_curseur
LOOP
...
END LOOP;
END;
```

Ecriture simplifiée

Exemple

Code PL/SQL	Commentaires
<pre>DECLARE CURSOR cur_emp IS SELECT empno, ename FROM emp WHERE deptno != 10;</pre>	Déclaration du curseur.
<pre>BEGIN FOR enreg IN cur_emp LOOP DBMS_OUTPUT.PUT_LINE('nom : ' enreg.ename ' (' enreg.empno ').'); END LOOP; END; /</pre>	Itération dans le curseur.